

Extending the Reach of Testing

Software Engineering

Retrieve data, decode pages, interact with a browser

Gregory S. DeLozier, PhD

Three different jobs

Tool	Job
Requests	Retrieve API data or static HTML
Beautiful Soup	Decode the retrieved HTML
WebDriver / ChromeDriver	Test interaction and dynamic pages

What does each observation let us claim?

Setup sidebar: macOS

In the chapter's code folder:

```
python3 -m venv .venv  
source .venv/bin/activate  
python -m pip install -r requirements.txt
```

Install Python 3 and Chrome. Selenium Manager obtains ChromeDriver when needed.

Setup sidebar: Windows

In Command Prompt, in code:

```
py -m venv .venv  
.venv\Scripts\activate.bat  
python -m pip install -r requirements.txt
```

```
PowerShell: .\venv\Scripts\Activate.ps1
```

Setup sidebar: Linux

Use the macOS environment commands.

Install Chrome or Chromium and its system dependencies.

Environment	Browser mode
Graphical desktop	Visible browser
No graphical desktop	Headless browser

Headless browsers still run JavaScript.

Running the examples

Use a terminal in the chapter's `code` folder, with the environment activated.

```
python -m pytest -v -s
```

`-v`: named pass/fail results. `-s`: show printed output.

Each example uses a `test_` function and assertions.

Live: get GitHub API data

```
url = "https://api.github.com/repos/python/cpython"
response = requests.get(url, timeout=15)
print(response.status_code)
repo = response.json()
print(repo["full_name"])
```

What else did GitHub return?

Run: `python -m pytest -v -s test_api.py`
Prints python/cpython and its description.

Live: choose useful checks

```
assert response.status_code == 200
assert repo["full_name"] == "python/cpython"
assert repo["private"] is False
```

Inspect the description, default branch, and star count.

Which values should stay fixed?

```
Run: python -m pytest -v -s test_api.py
```

Shows the test result and repository details.

Live: retrieve Wikipedia HTML

```
url = "https://en.wikipedia.org/wiki/Selenium_(software)"
response = requests.get(url, timeout=15)
assert response.status_code == 200
html = response.text
print(html[:500])
```

Same library. This time the response is HTML.

```
Run: python -m pytest -v -s test_wikipedia_response.py
Prints status 200 and the first 500 HTML characters.
```

Live: decode the HTML

```
soup = BeautifulSoup(html, "html.parser")
heading = soup.find(id="Selenium_WebDriver")
assert heading is not None
paragraph = heading.find_next("p")
assert paragraph is not None
print(paragraph.get_text(" ", strip=True))
```

Requests retrieved it. BeautifulSoup decoded it.

```
Run: python -m pytest -v -s test_static_html.py
Prints the WebDriver paragraph and its first link.
```

Live: examine a link

```
link = paragraph.find("a")
assert link is not None
print(link.get_text(" ", strip=True))
print(link.get("href"))
```

Does this establish that the destination works?

```
Run: python -m pytest -v -s test_static_html.py
Prints the first link text and destination.
```

Now interact with the page

Wikipedia search:

Type Selenium -> suggestions appear -> choose the software article

Selenium -> ChromeDriver -> Chrome

Which part requires the running browser?

Live: type into Wikipedia

```
browser.get("https://en.wikipedia.org/wiki/Main_Page")
search = wait.until(EC.element_to_be_clickable(
    (By.NAME, "search")))
search.send_keys("Selenium")
```

What appears after typing?

```
Run: python -m pytest -v -s test_wikipedia_webdriver.py
```

Full demo: opens Chrome, searches, prints text, closes.

Live: select the suggestion

```
suggestion = wait.until(software_suggestion)
suggestion.click()
```

The helper finds the visible **Selenium (software)** suggestion.

How long should the test wait for it?

```
Run: python -m pytest -v -s test_wikipedia_webdriver.py
```

The full demo selects Selenium (software).

Live: read the result

```
heading = browser.find_element(By.ID, "Selenium_WebDriver")
paragraph = heading.find_element(By.XPATH, "following::p[1]")
text = paragraph.text
print(text)
```

Did we reach the expected article and section?

Run: `python -m pytest -v -s test_wikipedia_webdriver.py`
Prints the WebDriver paragraph in the terminal.

What did we establish?

Observation	Evidence about
GitHub repository fields	API response data
Wikipedia paragraph in HTML	Delivered page content
Search suggestion appears	Dynamic browser behavior
Selecting it opens the article	Interactive navigation

A passing check supports a particular claim.

A final note about waiting

ChromeDriver: capable, implements the WebDriver standard.

Dynamic pages make timing awkward: what condition, and how long?

Playwright: automatic action waits and retrying assertions reduce that work.

Microsoft, “Auto-waiting”: <https://playwright.dev/python/docs/actionability>